SECTION 1 · SUBSECTION 1.3
ARCHITECTURE AND DEPLOYMENT

1.3.1 INTRODUCTION.

The system has one analysis engine and two delivery paths. The engine is a Python package under *src/* whose seven stages communicate only through an *ArchaeonState* dataclass. The delivery paths are a command-line runner, *run.py*, and a FastAPI service that stores job state and either dispatches to a Redis queue or runs the pipeline in a local thread. The single idea the reader must leave with is that the API layer does not contain analysis logic; it contains scheduling and status (figure 1.3-1).

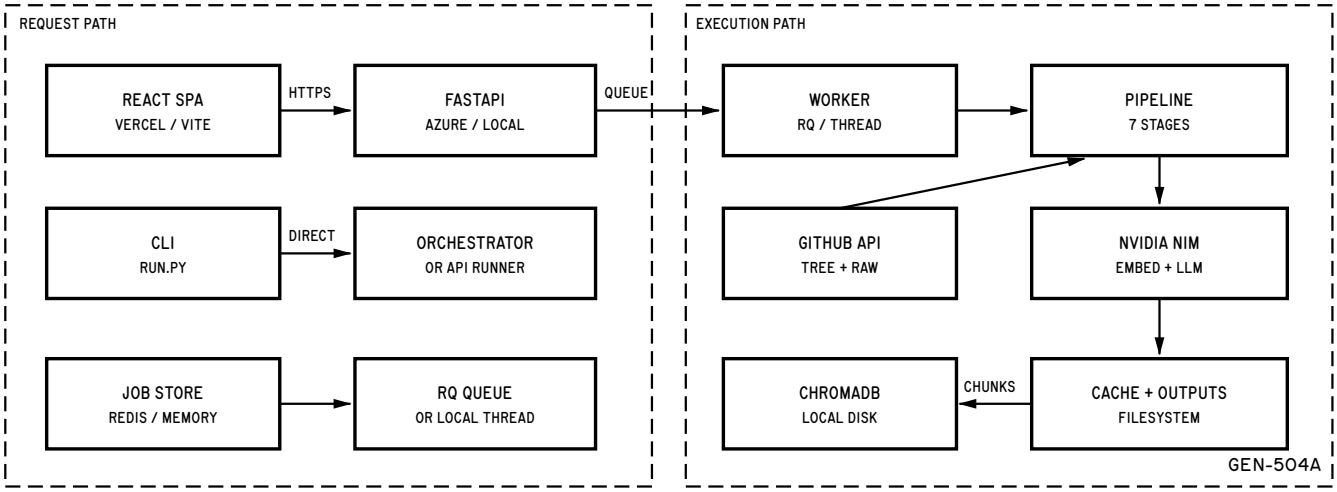


Figure 1.3-1. Runtime Components and Two Delivery Paths

1.3.2 FUNCTIONAL DESCRIPTION.

The API validates the URL and repository size, creates a job, then enqueues it when *REDIS_URL* is reachable or submits a local thread otherwise. Either runner constructs an *ArchaeonState*, calls the seven stages in sequence (fanning stages 3–5 out to three threads) and writes a status transition at each boundary. The SPA polls the job endpoint and renders the stored result; the CLI prints progress and writes files under *outputs/*.

1.3.2.1 State as the Only Contract.

No stage imports another stage. Each exposes *run(state) -> state*, reads named fields and writes named fields. The orchestrator is the only caller. This is what makes a stage testable in isolation and what allows stages 3, 4 and 5 to run concurrently without locks: they write disjoint fields.

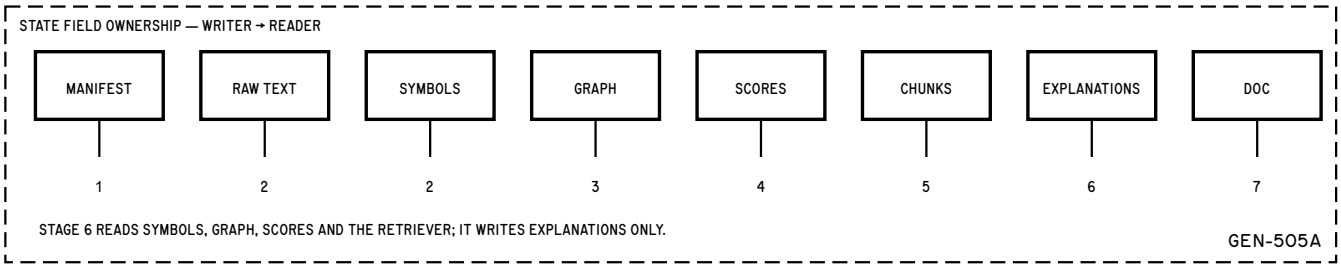


Figure 1.3-2. State Field Ownership by Stage

1.3.3 PERFORMANCE AND DESIGN DATA.

Table 1.3-1. Runtime Components

Component	Implementation	Responsibility
Display	React SPA	Submit, poll, render results
API	FastAPI + Uvicorn	Validate, deduplicate, schedule, report
Job store	Redis or dict	Status, result, active index
Queue	RQ or thread	Decouple submission from execution
Worker	Python module	Consume jobs, serve health on PORT
Engine	7 Python stages	Transform repository text to documents
Vector store	ChromaDB 1.5.9	Per-job chunk collection
External	GitHub, NVIDIA	Source text, embeddings, explanations

CAUTION

The local-thread path and the Redis path share *pipeline_runner.run*. If Redis is configured but the worker is not running, jobs remain queued indefinitely; the API does not detect a missing consumer. Confirm the worker before diagnosing a stuck job.

PATH SELECTION	OPERATOR
1 READ - REDIS_URL in environment	
2 PROBE - redis reachable	
√ SET - queue gnosis_jobs	
3 START - python -m src.api.worker	
4 VERIFY - job leaves queued	

GEN